



UNIVERSIDADE D
COIMBRA

Diogo Monteiro

DATA COMPRESSION:
LOSSY AND LOSSLESS

VOLUME 1

Work completed during the internship at Active Space
Technologies, supervised by Fernando Simões, and presented to
Active Space Technologies.

September 2025

Abstract

This report summarises the internship work on lossless and lossy data compression. It describes objectives, dataset organization, experimental methodology, results and conclusions.

Contents

1	Introduction	2
1.1	Background	2
1.2	Objective	2
2	Lossless Compression	3
2.1	Dataset	3
2.2	Measurements	3
2.3	Initial Selection	4
2.4	Brotli vs. LZ4 vs. Zstd	5
2.5	Results	5
3	Lossy Compression	9
3.1	Dataset	9
3.2	Procedures	9
3.3	Results	10
3.4	Plot Analysis	11
4	Conclusion	12
4.1	Summary	12

List of Tables

2.1	Lossless - Brotli and Zstd compression comparison on maximum levels. . .	5
2.2	Lossless - Demonstration of Zstd compression results across all levels in C++	6
2.3	Lossless - Time Comparison between Python and C++ while compressing using Zstd	7

List of Figures

3.1	Lossy - Objective visualisations for both 8 kbps and 128 kbps.	11
-----	--	----

Chapter 1

Introduction

1.1 Background

Data exist in diverse forms and scales. They may range from brief textual records, such as log files, to large-scale databases used to train contemporary large-language models (e.g., ChatGPT, Ollama) in industrial data centers. Even analog phenomena like sound waves can be represented and analyzed as sequences of data points.

1.2 Objective

The objective of this work is to investigate both lossless and lossy data compression algorithms, evaluate their performance across representative scenarios, determine which methods are most efficient for different data types while also comparing performance between high-level and low-level programming languages namely Python and C++ for compression. This evaluation is particularly relevant for telemetry applications, where effective compression reduces storage requirements, accelerates data transmission, and lowers operational costs.

Chapter 2

Lossless Compression

2.1 Dataset

We organize the dataset into multiple folders inside a main folder:

- Lossless (main)
 - Data;
 - Excel;
 - PDF2TXT;
 - Photos;
 - Text.

2.2 Measurements

2.2.1 Compression

- Compression ratio (percentage);
- Compression time (seconds);
- Compression per Time Index (CTI)

$$\text{CTI} = \frac{\text{Compression Ratio (\%)}}{\text{Compression Time (s)}}$$

2.2.2 Decompression

- Decompression time (seconds).

2.2.3 Compression and Decompression

- Central Processing Unit/CPU usage (percentage).
- Memory usage (bytes).

2.3 Initial Selection

To gain a better understanding on the performance of each method in terms of compression, we observe the following methods behaviour according to the metrics mentioned in **Section 2.2.1**.

2.3.1 Methods

- **Gzip**: Based on the DEFLATE algorithm (LZ77 and Huffman coding). It provides a traditional balance between compression ratio and speed, being widely used for general file and web compression.
Reference: <https://www.gnu.org/software/gzip/>
- **Brotli**: Developed by Google, it combines a modern variant of LZ77, Huffman coding, and 2nd-order context modeling. It achieves higher compression ratios than Gzip, particularly for text, while maintaining fast decompression speeds.
Reference: <https://github.com/google/brotli>
- **Zstd (Zstandard)**: Developed by Meta, it uses Finite State Entropy (FSE) and LZ77. It offers compression ratios comparable to traditional methods but with significantly faster compression and decompression speeds, scalable across multiple levels.
Reference: <https://github.com/facebook/zstd>
- **BZ2 (bzip2)**: Utilizes the Burrows-Wheeler Transform (BWT) alongside Move-to-Front and Huffman coding. It prioritizes high compression ratios over speed, resulting in longer processing times.
Reference: <https://sourceware.org/bzip2/>
- **LZMA**: Standing for Lempel-Ziv-Markov chain algorithm, it achieves exceptionally high compression ratios. It requires significant memory and computational time to compress, though decompression remains relatively fast.
Reference: <https://tukaani.org/xz/>
- **LZ4**: A dictionary-based algorithm (LZ77) explicitly designed for extreme speed. It achieves compression and decompression speeds in the GB/s range, trading maximum compression ratio for near-zero latency.
Reference: <https://github.com/lz4/lz4>

2.3.2 Observations

Every method has their benefits but the ones that stand out the most are:

- Brotli, since often gives the highest compression rates.
- LZ4, which is extremely fast.
- Zstd, that shows a very good trade-off (high CTI).

2.4 Brotli vs. LZ4 vs. Zstd

To bring the study further, we investigate Brotli, LZ4 and Zstd across all available compression levels since these methods are the ones with best results. Measurement details in **Section 2.2** and subsections.

2.5 Results

2.5.1 Best Method

The experimental results indicate that compression algorithms present distinct trade-offs between achieved compression rates and processing time. Most modern algorithms allow users to tune this trade-off via compression “levels”. A higher level dictates that the algorithm will expend more computational effort (CPU cycles and memory) to find data redundancies, thereby maximizing the compression ratio at the severe expense of processing speed.

To establish the upper bounds of performance, we first compared Brotli and Zstd at their respective maximum compression settings: Level 11 for Brotli and Level 22 for Zstd. As demonstrated in Table 2.1, Brotli attains marginally higher compression ratios, but requires substantially more computational time compared to Zstd’s maximum effort.

Table 2.1: Lossless - Brotli and Zstd compression comparison on maximum levels.

Algorithm	Level	Compression time (s)	Compression ratio (%)
Brotli	11	288	81.05
Zstd	22	146	80.31

Conversely, algorithms like LZ4 deliver exceptionally fast processing times, but yield significantly lower compression ratios when compared to Brotli and Zstd, even at conservative settings.

Given Zstd’s favorable baseline performance, we conducted a comprehensive sweep across all its 22 levels to map its efficiency curve and identify an optimal operating point. Table 2.2 details these results, highlighting the Compression per Time Index (CTI).

The data reveals that operating Zstd at very high levels (notably above level 19) yields diminishing returns in data reduction while requiring substantially increased CPU and memory resources. On resource-limited machines, these extreme settings occasionally cause execution failures or unacceptably long processing times.

Therefore, we recommend adopting low to moderate Zstd levels—namely 1, 8, or at a maximum 16—to achieve a pragmatic balance between compression ratios and available computing capabilities.

Table 2.2: Lossless - Demonstration of Zstd compression results across all levels in C++

Level	Compression rate (%)	Time (s)	Compression per Time Index
Zstd Lv.01	73.93	0.28	265.94
Zstd Lv.02	74.01	0.35	214.53
Zstd Lv.03	72.99	0.48	151.11
Zstd Lv.04	73.32	0.58	126.63
Zstd Lv.05	74.45	1.12	66.41
Zstd Lv.06	74.98	1.46	51.36
Zstd Lv.07	75.44	1.63	46.22
Zstd Lv.08	75.73	2.01	37.77
Zstd Lv.09	76.04	2.75	27.69
Zstd Lv.10	76.38	2.81	27.19
Zstd Lv.11	76.50	3.79	20.21
Zstd Lv.12	76.52	4.03	19.01
Zstd Lv.13	76.63	10.77	7.11
Zstd Lv.14	76.72	12.27	6.25
Zstd Lv.15	76.81	17.65	4.35
Zstd Lv.16	78.60	25.30	3.11
Zstd Lv.17	79.16	34.37	2.30
Zstd Lv.18	79.51	46.29	1.72
Zstd Lv.19	80.14	79.10	1.01
Zstd Lv.20	80.27	89.19	0.90
Zstd Lv.21	80.32	98.16	0.82
Zstd Lv.22	80.31	146.25	0.55

2.5.2 Python vs C++

The implementations in Python and C++ exhibit similar compression ratios; however, they present distinct trade-offs regarding execution time, resource allocation, and memory stability. To ensure a fair comparison, all benchmarks were executed on the same hardware under identical baseline conditions.

In terms of speed, the C++ implementation demonstrated a clear advantage. The total elapsed compression time for the dataset in Python was 22 min (1320 s), whereas the corresponding C++ execution required 18 min (1080 s). This represents a reduction in compression time of approximately 18.2%, directly improving the Compression per Time Index (CTI) in favour of C++. Decompression followed a similar trend, being approximately 30% faster in C++, although both languages achieved decompression times well below 1 s in the tested cases. Average CPU utilisation remained comparable between the two environments ($\sim 5\%$ for Python and $\sim 8\%$ for C++).

Table 2.3 details the specific time differences across all Zstd compression levels, highlighting the growing performance gap as the computational complexity increases.

Table 2.3: Lossless - Time Comparison between Python and C++ while compressing using Zstd

Zstd Level	Time with Python (s)	Time with C++ (s)
1	0.36	0.27
2	0.46	0.32
3	0.66	0.46
4	0.78	0.58
5	1.33	1.06
6	1.79	1.39
7	2.06	1.55
8	2.56	1.88
9	2.83	2.06
10	3.82	2.54
11	5.18	3.24
12	5.58	3.47
13	10.27	9.09
14	12.10	12.27
15	17.18	17.34
16	26.13	25.87
17	35.31	33.90
18	52.29	45.74
19	87.91	79.81
20	103.03	86.89
21	115.61	97.17
22	330.81	146.28

Beyond execution time, the two languages exhibit fundamentally different memory management behaviors. Python consumes less memory at lower compression levels, but

its dynamic allocation leads to a substantial increase in peak memory usage at higher levels (especially with Zstd). Conversely, the C++ algorithms pre-allocate and reserve memory during the compression process. While this directly influences the speed difference, it also guarantees that memory consumption remains constant and predictable, unlike Python.

Recommendation: Based on the criteria of execution speed and memory predictability, C++ is the superior choice for deploying these compression algorithms. While Python offers advantages for rapid prototyping, C++ eliminates dynamic memory overhead and provides the robust, deterministic performance required for resource-constrained environments like telemetry systems.

Chapter 3

Lossy Compression

3.1 Dataset

- Periodic functions, as .txt files (one column, 1 000 000 samples).
- Musical sample: *Beethoven — Fifth Symphony*.

3.2 Procedures

3.2.1 Periodic functions

1. Extract and convert numerical sequences from .text to .wav using MATLAB for direct codec processing.
2. Encode the reference .wav to Opus at the target bitrates using a C++ script.
3. Perform subjective listening comparisons against the original reference.

It is hard to hear the differences on these sounds since they are not diversified on an audio level so we decide to proceed with a music study.

3.2.2 Music Sample

1. Get the .wav file for the music.
2. Encode audio .wav file to Opus at the target bitrates using a C++ script. Namely 128, 64, 32, 8, 1 kbps.
3. Perform subjective listening comparisons against the original reference.
4. Compute objective metrics and visualisations (time-domain waveform, spectrogram, magnitude spectrum) in MATLAB for 8, 32 and 128 kbps.

3.3 Results

3.3.1 Subjective results

Informal listening tests with headphones produced the following qualitative ranking relative to the original:

- 128 kbps: effectively transparent in informal tests (reported $\approx 91\%$ size reduction in the reported run).
- 64 kbps: small but perceptible differences.
- 32 kbps: noticeable degradation in fidelity.
- 8 kbps: clearly degraded; listening fatigue increases.
- 1 kbps: effectively noise for musical content.

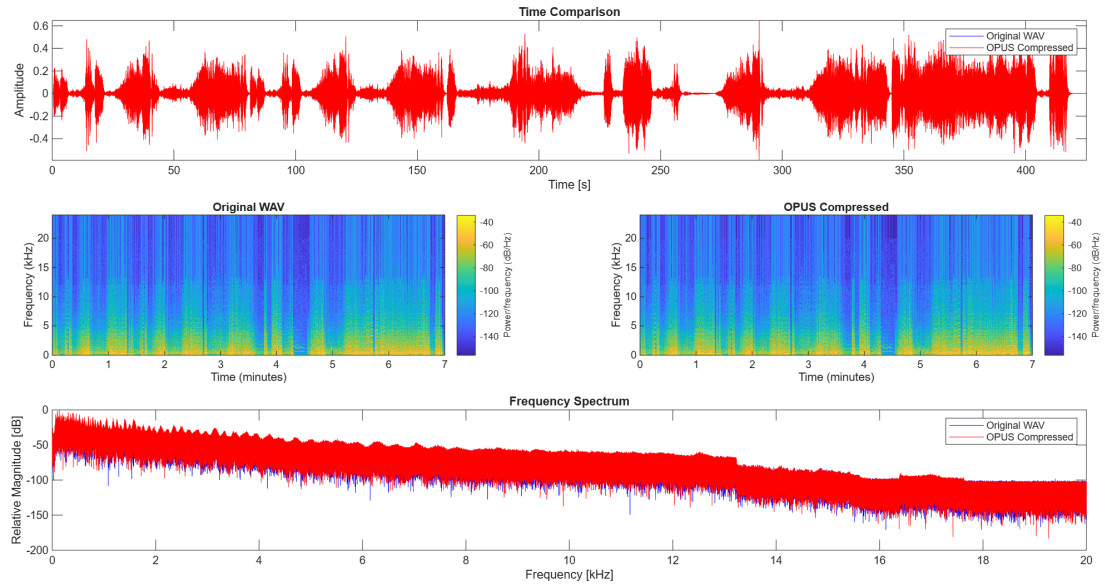
3.3.2 Objective analysis

For objective comparison, as mentioned on **Section 3.2.2** we observe that:

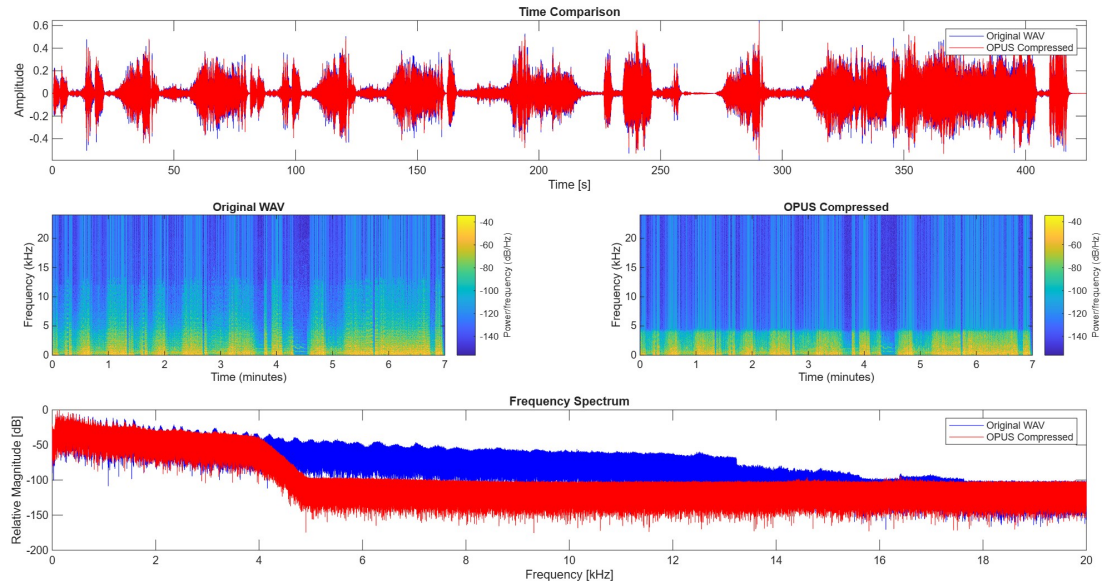
- 8 kbps exhibits strong attenuation of upper bands; spectrograms show pronounced reduction above $\sim 3\text{--}5\text{ kHz}$ and the magnitude spectrum displays a clear low-pass behaviour.
- 32 kbps shows the same trends with milder attenuation.
- 128 kbps closely matches the reference in spectral content; residual differences are limited to low-amplitude components.

3.4 Plot Analysis

Here we have a better visualization of the results.



(a) 128 kbps time-domain waveform, spectrogram, magnitude spectrum



(b) 8 kbps time-domain waveform, spectrogram, magnitude spectrum

Figure 3.1: Lossy - Objective visualisations for both 8 kbps and 128 kbps.

Chapter 4

Conclusion

4.1 Summary

To conclude for lossless methods, low-to-moderate levels of Zstd provides the best trade-off between compression ratio, runtime and resource consumption. For lossy audio, Opus at 128 kbps delivers near-transparent perceptual quality while yielding substantial file-size reduction. C++ implementations consistently outperformed Python in execution time and exhibited more predictable memory behaviour.

With all of these in mind, our selection should prioritise an optimal operating point per use case rather than maximal compression.

4.2 References

- **Source Code & Results:** Monteiro, D. *AST-Data-Compression*. GitHub repository. Available at: <https://github.com/monteiro-diogo/AST-Data-Compression>